Jawaban UTS Praktikum PBO

Nama : Torikh Abdullah Naser
NIM : 2410512031
Kelas : A-S1 Sistem Informasi

Soal 1

a. Perbedaan Paradigma Prosedural dan OOP dalam Sistem Informasi Manajemen Karyawan

- **Paradigma Prosedural:** Pendekatan ini berfokus pada urutan langkah atau fungsi (prosedur) untuk memanipulasi data karyawan. Dalam sistem ini, struktur data (seperti array/record karyawan) dan fungsi (seperti hitungGaji) dibuat secara terpisah. Ketika sistem manajemen karyawan menjadi kompleks, pemisahan ini bisa membuat kode sulit dikelola dan rentan terhadap kesalahan (karena fungsi apa saja berpotensi mengubah data karyawan).
- **Paradigma OOP (Object-Oriented Programming):** Pendekatan ini berfokus pada **objek** sebagai entitas utama yang menggabungkan data (atribut) dan perilaku (metode) dalam satu kesatuan (Enkapsulasi). Dalam sistem karyawan, kita dapat merancang *blueprint* berupa class Karyawan. Hal ini membuat kode lebih modular, data lebih terlindungi dari modifikasi sembarangan, dan sangat mendukung pengembangan lebih lanjut (misal penambahan Karyawan Tetap, Karyawan Kontrak) melalui konsep pewarisan (Inheritance) dan polimorfisme.

b. Identifikasi Komponen OOP pada Sistem

- **Class:** Karyawan (Berfungsi sebagai *blueprint* atau cetakan dasar yang mendefinisikan informasi dan kemampuan yang harus dimiliki oleh setiap pekerja).
- **Object:** Instansiasi atau wujud nyata dari class Karyawan di dalam memori saat program berjalan. Contohnya, objek karyawan1 (mewakili data Budi) dan objek karyawan2 (mewakili data Siti).
- **Attribute (State):** Variabel penyimpan data atau karakteristik yang dimiliki oleh karyawan. Contoh: idKaryawan, nama, jabatan, gajiPokok.
- **Method (Behavior):** Fungsi yang mendefinisikan aksi atau operasi yang bisa dilakukan oleh atau pada objek karyawan tersebut. Contoh: tampilkanInfo(), hitungGajiTotal(), setJabatan().

c. Rancangan Struktur Sederhana Class Karyawan

```java
public class Soal1_Karyawan {
    private String idKaryawan;
    private String nama;
    private String jabatan;
    private double gajiPokok;

    public Soal1_Karyawan(String idKaryawan, String nama, String jabatan, double gajiPokok) {
        this.idKaryawan = idKaryawan;
        this.nama = nama;
        this.jabatan = jabatan;
        this.gajiPokok = gajiPokok;
    }

    public void tampilkanInfo() {
        System.out.println("ID Karyawan : " + this.idKaryawan);
        System.out.println("Nama        : " + this.nama);
        System.out.println("Jabatan     : " + this.jabatan);
        System.out.println("Gaji Pokok  : Rp" + this.gajiPokok);
        System.out.println("-----------------------------------------------");
    }

    public double hitungGajiTotal() {
        double tunjangan = 0.1 * this.gajiPokok;
        return this.gajiPokok + tunjangan;
    }

    Run | Debug
    public static void main(String[] args) {
        System.out.println("====== SISTEM INFORMASI MANAJEMEN KARYAWAN ======\n");

        Soal1_Karyawan karyawan1 = new Soal1_Karyawan("K001", "Budi Santoso", "Manager", 8000000);
        Soal1_Karyawan karyawan2 = new Soal1_Karyawan("K002", "Siti Aminah", "Staff IT", 5000000);

        System.out.println("--- Data Karyawan ---");
        karyawan1.tampilkanInfo();
        karyawan2.tampilkanInfo();

        System.out.println("\n--- Perhitungan Gaji Pokok + Tunjangan ---");
        System.out.println("Total Gaji " + karyawan1.nama + " : Rp" + karyawan1.hitungGajiTotal());
        System.out.println("Total Gaji " + karyawan2.nama + " : Rp" + karyawan2.hitungGajiTotal());
    }
}
```

```
====== SISTEM INFORMASI MANAJEMEN KARYAWAN ======

--- Data Karyawan ---
ID Karyawan : K001
Nama        : Budi Santoso
Jabatan     : Manager
Gaji Pokok  : Rp8000000.0
-----------------------------------------------
ID Karyawan : K002
Nama        : Siti Aminah
Jabatan     : Staff IT
Gaji Pokok  : Rp5000000.0
-----------------------------------------------

--- Perhitungan Gaji Pokok + Tunjangan ---
Total Gaji Budi Santoso : Rp8800000.0
Total Gaji Siti Aminah : Rp5500000.0
```

Analisis Kode: Pada implementasi class Soal1_Karyawan, kita menerapkan konsep dasar OOP dengan mendefinisikan *attribute* dengan hak akses private (seperti idKaryawan, nama, jabatan, dan gajiPokok) untuk melindungi data internal. Kita juga menyediakan *constructor* untuk menginisialisasi nilai awal secara terstruktur saat objek dibuat. Selain itu, terdapat

method tampilkanInfo() untuk menampilkan data karyawan dan hitungGajiTotal() untuk merangkum logika perhitungan gaji beserta tunjangan. Hal ini menunjukkan penerapan *Enkapsulasi* di mana status (data) dan perilaku (fungsi) digabungkan dan dikelola di dalam satu entitas objek yang utuh.

Soal 2

1. Analisis Kelemahan Kode dari Sisi Keamanan Data dan Kode Etik Penggunaan Data

Kode awal:

```
public class Mahasiswa {
   public String nama;
   public int nim;
}
```

- **Keamanan Data:** Menggunakan *access modifier* public pada atribut membuat data tersebut terbuka dan bisa diakses serta dimodifikasi secara langsung dari luar class tanpa ada validasi (kontrol). Hal ini sangat berbahaya karena program lain bisa mengubah nilai nim menjadi negatif, *null*, atau format yang salah, sehingga merusak integritas (konsistensi) data dalam sistem.
- **Kode Etik Penggunaan Data:** Data seperti nama dan nim merupakan data pribadi (Personally Identifiable Information/PII). Membiarkan data sensitif ini terekspos secara publik melanggar prinsip hak akses minimal (*least privilege*). Dalam desain sistem yang baik, data pribadi harus dilindungi dan dijaga kerahasiaannya agar tidak mudah disalahgunakan atau dibaca oleh komponen/class lain yang tidak berhak.

2. Perbaikan Kode Menggunakan Konsep Encapsulation

```java
public class Soal2_Mahasiswa {
    private String nama;
    private int nim;

    public Soal2_Mahasiswa(String nama, int nim) {
        this.nama = nama;
        this.nim = nim;
    }

    public String getNama() {
        return nama;
    }

    public void setNama(String nama) {
        if (nama != null && !nama.trim().isEmpty()) {
            this.nama = nama;
        } else {
            System.out.println("Error: Nama tidak boleh kosong.");
        }
    }

    public int getNim() {
        return nim;
    }
    public void setNim(int nim) {
        if (nim > 0) {
            this.nim = nim;
        } else {
            System.out.println("Error: NIM harus bernilai positif.");
        }
    }

    Run | Debug
    public static void main(String[] args) {
        Soal2_Mahasiswa mhs1 = new Soal2_Mahasiswa("Torikh", 241051203);

        System.out.println("--- Data Awal ---");
        System.out.println("Nama : " + mhs1.getNama());
        System.out.println("NIM  : " + mhs1.getNim());

        System.out.println("\n--- Uji Validasi Setter ---");
        mhs1.setNama("");
        mhs1.setNim(-123);

        System.out.println("\n--- Data Setelah Perubahan ---");
        mhs1.setNama("Torikh Abdullah Naser");
        mhs1.setNim(241051203);
        System.out.println("Nama : " + mhs1.getNama());
        System.out.println("NIM  : " + mhs1.getNim());
    }
}
```

```
--- Data Awal ---
Nama : Torikh
NIM  : 241051203

--- Uji Validasi Setter ---
Error: Nama tidak boleh kosong.
Error: NIM harus bernilai positif.

--- Data Setelah Perubahan ---
Nama : Torikh Abdullah Naser
NIM  : 241051203
```

Analisis Kode: Pada perbaikan class Soal2_Mahasiswa, seluruh atribut (nama dan nim) diubah aksesnya menjadi private agar tidak bisa diakses dan diedit sembarangan secara langsung dari luar class (Data Hiding). Untuk berinteraksi dengan data tersebut, kita menyediakan *method*

getter (seperti getNama() dan getNim()) yang berfungsi hanya untuk membaca nilai, serta *method setter* (seperti setNama() dan setNim()) untuk mengubah nilainya. Di dalam *method setter* tersebut, disisipkan logika validasi (misal, nama tidak boleh kosong dan nim harus bernilai lebih dari 0) sehingga sistem bisa menolak input yang tidak valid dan mencegah kerusakan integritas data.

3. Manfaat Penerapan Encapsulation dalam Sistem Enterprise

- **Data Hiding & Security (Keamanan Data):** Menyembunyikan detail representasi data di dalam objek. Atribut dibuat private sehingga tidak bisa diakses langsung, melindungi data internal dari modifikasi yang tidak disengaja atau tidak sah.
- **Validation & Control (Validasi dan Kontrol):** Dengan menggunakan *setter*, kita dapat menambahkan logika validasi aturan bisnis sebelum mengubah data. Misalnya, memastikan format NIM sudah benar atau memastikan nama tidak boleh kosong.
- **Maintainability & Flexibility (Kemudahan Pemeliharaan):** Implementasi internal suatu class dapat diubah sewaktu-waktu (misal cara menyimpan data nim diubah dari int menjadi String di level *database*) tanpa merusak kode program lain yang memanggil *getter/setter*-nya, selama *interface method* tidak berubah.
- **Read-Only / Write-Only Properties:** Encapsulation memungkinkan pengembang untuk menentukan kontrol akses yang lebih presisi (misalnya hanya membuat metode *getter* agar data bersifat *read-only* bagi komponen luar).

Soal 3

1. Konsep Inheritance dan Overriding pada Kasus Person & Employee

- **Inheritance (Pewarisan):** Dalam kasus ini, Employee adalah class turunan (*subclass/child class*) dari Person (*superclass/parent class*). Ini berarti Employee akan secara otomatis mewarisi atribut (nama, umur) dan *method* dari Person. Konsep ini mencegah duplikasi kode, karena kita tidak perlu mendeklarasikan ulang atribut nama dan umur pada class Employee, cukup menambahkan atribut spesifik seperti idKaryawan.
- **Overriding:** Terjadi ketika *subclass* (Employee) mendeklarasikan ulang atau menimpa *method* yang sudah ada di *superclass* (Person) dengan *signature* (nama method dan parameter) yang sama persis. Contohnya, jika Person memiliki method tampilkanInfo() untuk mencetak nama dan umur, maka Employee dapat melakukan *override* pada method tersebut agar selain mencetak nama dan umur, ia juga turut mencetak idKaryawan.

2. Implementasi Kode Java (Inheritance & Overriding)

```java
class Soal3_Person {
    protected String nama;
    protected int umur;

    public Soal3_Person(String nama, int umur) {
        this.nama = nama;
        this.umur = umur;
    }

    public void tampilkanInfo() {
        System.out.println("Nama : " + this.nama);
        System.out.println("Umur : " + this.umur + " tahun");
    }
}

class Soal3_Employee extends Soal3_Person {
    private String idKaryawan;

    public Soal3_Employee(String nama, int umur, String idKaryawan) {
        super(nama, umur);
        this.idKaryawan = idKaryawan;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("ID   : " + this.idKaryawan);
    }
}

public class Soal3_SistemManajemen {
    Run | Debug
    public static void main(String[] args) {
        System.out.println("=== Data Person ===");
        Soal3_Person p1 = new Soal3_Person("Agus", 45);
        p1.tampilkanInfo();

        System.out.println("\n=== Data Employee ===");
        Soal3_Employee e1 = new Soal3_Employee("Torikh", 20, "EMP-2026-001");
        e1.tampilkanInfo();
    }
}
```

```
=== Data Person ===
Nama : Agus
Umur : 45 tahun

=== Data Employee ===
Nama : Torikh
Umur : 20 tahun
ID   : EMP-2026-001
PS D:\kuliah\smt 4\prak pbo>
```

Analisis Kode: Pada implementasi di Soal3_SistemManajemen.java, kita mendefinisikan class Person dengan atribut nama dan umur serta method tampilkanInfo(). Kemudian, class Employee menggunakan kata kunci extends Person untuk mewarisi sifat Person. Employee memiliki atribut tambahannya sendiri yaitu idKaryawan. Konstruktor Employee memanggil super(nama, umur) untuk menginisialisasi atribut induknya. Kita juga menggunakan anotasi @Override pada tampilkanInfo() di class Employee untuk menandakan bahwa kita memperluas perilaku method tersebut dari perilaku aslinya di Person.

3. Analisis Keuntungan Penggunaan Inheritance dalam Sistem Enterprise

- **Reusability (Daur Ulang Kode):** Penulisan logika umum yang hanya dilakukan satu kali pada *superclass* (misal logika Person) membuat *subclass* (seperti Employee, Pelanggan, Vendor) dapat langsung memakainya. Ini sangat mengurangi redundansi baris kode pada program enterprise yang memiliki ratusan entitas.

- **Maintainability (Kemudahan Pemeliharaan):** Jika terdapat perbaikan (*bug fix*) atau perubahan aturan pada level struktur data umum, pengembang hanya perlu mengubahnya di satu tempat (pada *superclass*). Perubahan tersebut secara otomatis akan diwariskan ke semua *subclass*, mencegah inkonsistensi.
- **Extensibility (Skalabilitas Tinggi):** Sistem enterprise sering berkembang. Jika suatu hari perusahaan ingin menambah jabatan yang lebih spesifik (seperti class Manager atau Director), developer cukup membuat class baru yang mewarisi class Employee tanpa perlu membangun segalanya dari nol.
- **Mendukung Polymorphism:** Inheritance adalah tulang punggung dari *Polymorphism*. Di sistem besar, hal ini memungkinkan array atau *collection* tipe data induk (Person[]) untuk menyimpan berbagai macam tipe data turunan (Employee, dll.) dan memanipulasinya secara seragam (misal dalam satu perulangan for), sehingga kode arsitektur bisnis menjadi lebih abstrak dan rapi.

Soal 4

1. Analisis Kebutuhan User dan Tujuan Organisasi

- **Kebutuhan User (Petugas/Pustakawan):** Membutuhkan antarmuka atau sistem yang mudah untuk mencatat data buku baru, melakukan pencatatan transaksi peminjaman buku, dan mencetak laporan sirkulasi dengan cepat tanpa harus menghitung secara manual.
- **Tujuan Organisasi:** Melakukan digitalisasi inventaris untuk mencegah kehilangan aset buku, memonitor tingkat sirkulasi peminjaman sebagai tolak ukur minat baca, serta menyediakan laporan yang akurat secara *real-time* untuk keperluan audit dan evaluasi kinerja perpustakaan.

2. Rancang Struktur Class dan Method

Berdasarkan spesifikasi, berikut adalah rancangan desain berorientasi objeknya:

- **Class Buku:**
  - *Attribute:* judul, pengarang, statusDipinjam
  - *Method:* getJudul(), isDipinjam(), setDipinjam()
- **Class Perpustakaan (Sebagai sistem utamanya):**
  - *Attribute:* daftarBuku (List/Array), jumlahPinjaman (int)
  - *Method:*
    - tambahBuku(Buku b) -> untuk menyimpan data buku.
    - pinjamBuku(Buku b) -> untuk mengubah status buku dan merekap transaksi peminjaman.
    - hitungJumlahPinjaman() -> mengembalikan (return) nilai integer total pinjaman.
    - tampilkanLaporan() -> mencetak data rekap perpustakaan ke layar.

3. Implementasi Minimal 1 Class dengan Method (Void dan Non-Void)

```java
import java.util.ArrayList;
import java.util.List;

class Buku {
    private String judul;
    private boolean isDipinjam;

    public Buku(String judul) {
        this.judul = judul;
        this.isDipinjam = false;
    }

    public String getJudul() {
        return judul;
    }

    public boolean isDipinjam() {
        return isDipinjam;
    }

    public void setDipinjam(boolean status) {
        this.isDipinjam = status;
    }
}

public class Soal4_Perpustakaan {
    private List<Buku> daftarBuku;

    public Soal4_Perpustakaan() {
        this.daftarBuku = new ArrayList<>();
    }

    public void tambahBuku(Buku buku) {
        daftarBuku.add(buku);
    }
    public void pinjamBuku(Buku buku) {
        if (!buku.isDipinjam()) {
            buku.setDipinjam(true);
            System.out.println("-> Berhasil meminjam buku: " + buku.getJudul());
        } else {
            System.out.println("-> Gagal: Buku " + buku.getJudul() + " sedang dipinjam.");
        }
    }

    public int hitungJumlahPinjaman() {
        int jumlahPinjaman = 0;
        for (Buku buku : daftarBuku) {
            if (buku.isDipinjam()) {
                jumlahPinjaman++;
            }
        }
        return jumlahPinjaman;
    }

    public void tampilkanLaporan() {
        System.out.println("\n=== LAPORAN PERPUSTAKAAN DIGITAL ===");
        System.out.println("Total Buku Terdaftar : " + daftarBuku.size());

        System.out.println("Total Buku Dipinjam  : " + hitungJumlahPinjaman());
        System.out.println("Daftar Buku:");
        for (Buku buku : daftarBuku) {
            String status = buku.isDipinjam() ? "[Dipinjam]" : "[Tersedia]";
            System.out.println("- " + buku.getJudul() + " " + status);
        }
        System.out.println("==================================\n");
    }

    Run | Debug
    public static void main(String[] args) {
        Soal4_Perpustakaan perpus = new Soal4_Perpustakaan();

        Buku buku1 = new Buku("Pemrograman Java OOP");
        Buku buku2 = new Buku("Struktur Data & Algoritma");
        Buku buku3 = new Buku("Basis Data Relasional");

        perpus.tambahBuku(buku1);
        perpus.tambahBuku(buku2);
        perpus.tambahBuku(buku3);

        perpus.pinjamBuku(buku1);
        perpus.pinjamBuku(buku3);
        perpus.pinjamBuku(buku1);

        perpus.tampilkanLaporan();
    }
}
```

```
PS D:\kuliah\smt 4\prak pbo>  d:; cd 'd:\kuliah\smt 4\prak
sInExceptionMessages' '-cp' 'C:\Users\USER\AppData\Roaming
66d2\bin' 'Soal4_Perpustakaan'
-> Berhasil meminjam buku: Pemrograman Java OOP
-> Berhasil meminjam buku: Basis Data Relasional
-> Gagal: Buku Pemrograman Java OOP sedang dipinjam.

=== LAPORAN PERPUSTAKAAN DIGITAL ===
Total Buku Terdaftar : 3
Total Buku Dipinjam  : 2
Daftar Buku:
- Pemrograman Java OOP [Dipinjam]
- Struktur Data & Algoritma [Tersedia]
- Basis Data Relasional [Dipinjam]
====================================

PS D:\kuliah\smt 4\prak pbo>
```

Analisis Kode: Pada implementasi Soal4_Perpustakaan.java, dibuat sebuah class Perpustakaan yang bertugas me-*manage* objek Buku. Di dalamnya terdapat implementasi *method* bertipe **void** (seperti tambahBuku() dan tampilkanLaporan()) yang berfungsi untuk melakukan suatu aksi perubahan *state* atau mencetak output tanpa mengembalikan suatu nilai ke pemanggilnya. Di sisi lain, terdapat juga *method* bertipe **non-void** yaitu hitungJumlahPinjaman() yang memiliki *return type* int, berfungsi mengembalikan nilai kalkulasi matematis (jumlah buku yang berstatus sedang dipinjam) agar dapat diolah lebih lanjut oleh sistem.

4. Evaluasi Efektivitas dan Efisiensi Desain

- **Efektif:** Desain ini sudah sangat efektif karena secara langsung menjawab 3 *requirement* utama organisasi: fungsi penyimpanan terakomodasi di dalam ArrayList<Buku> dan tambahBuku(), fungsi kalkulasi terpenuhi melalui perulangan di hitungJumlahPinjaman(), dan fungsi rekapan tersedia di tampilkanLaporan(). Pemisahan entitas antara Buku dan Perpustakaan (prinsip *Single Responsibility*) membuat kode terstruktur dengan baik.
- **Efisien:** Penggunaan tipe kembalian (void dan non-void) diterapkan pada konteks yang tepat (tidak memboroskan alokasi nilai *return* jika hanya untuk mencetak). Penggunaan *Collection* seperti ArrayList lebih efisien daripada *Array* statis tradisional, karena ukurannya bisa bertambah secara dinamis sesuai jumlah buku, menghemat pemakaian memori sejak awal program berjalan.